IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received Month Day, 2026, date of publication Month Day, 2026, date of current version April 23, 2026.

Digital Object Identifier 10.1109/ACCESS.????.???????

Neural ODE based modeling of memristive devices

KAROL BEDNARZ[1], BARTŁOMIEJ GARDA[1], and ZBIGNIEW GALIAS[1], (Senior Member, IEEE)
[1]Department of Electrical Engineering, AGH University of Kraków, al. Mickiewicza 30, 30–059, Kraków, Poland

Corresponding author: Karol Bednarz (e-mail: kbednarz@agh.edu.pl).

This work was supported by the National Science Centre, Poland under Grant 2024/53/B/ST7/03841.

ABSTRACT The development of modern memristive technologies requires accurate dynamic models to enable reliable simulation and prototyping in applications such as resistive RAM, neural networks, and neuromorphic computing. Existing symbolic models often exhibit limitations in accurately capturing the complex dynamic behavior of physical devices across various operating conditions, impacting their reliability in simulations. This work proposes a neural-network-based modeling of real memristors behaviour using Neural Ordinary Differential Equation (Neural ODE) models. Prior neural-network-based studies were limited by training data derived solely from theoretical models, preventing reliable assessment on real devices. The proposed method leverages a backpropagation technique based on solving the corresponding adjoint ODE backward in time using adjoint sensitivity methods. Experimental data obtained directly from Self-Directed Channel (SDC) memristors are used to train and validate the model. Results show that the Neural ODE approach more accurately reproduces the device's dynamic characteristics and provides improved flexibility and robustness compared to traditional theoretical models. These findings demonstrate the feasibility of Neural ODE–based frameworks for data-driven modeling of physical memristive devices, validated on a tungsten-doped Self-Directed Channel (SDC) device.

INDEX TERMS Memristor modeling, artificial neural network, neural ODE.

I. INTRODUCTION

NUMEROUS The use of PARstart is described in the template memristor models have been proposed in the scientific literature following the discovery of memristive behavior at the nanoscale. The original model, introduced in [1], conceptualizes the memristor as a series connection of two variable resistances corresponding to the conductive and insulating regions of the thin film.

Yakopcic et al. [2] developed a generalized memristor device model that captures the hyperbolic sine I-V relationship arising from metal-insulator-metal (MIM) structures, while incorporating voltage-dependent programming thresholds and polarity-asymmetric dynamics. In [3], the authors proposed the *TEAM* (ThrEshold Adaptive Memristor) model, which introduces a threshold-based approach to capture the switching behavior of memristors. The model defines distinct voltage thresholds for the SET and RESET operations, allowing for more accurate representation of the device's nonlinear dynamics during state transitions. In [4], the *VTEAM* (Voltage ThrEshold Adaptive Memristor) model extends the TEAM framework by incorporating voltage-dependent dynamics, enabling improved modeling of memristor behavior under varying voltage conditions. Both models have been widely adopted for simulating memristive devices in various applications.

Among models widely used for practical memristive devices is the Stanford model [5]–[8], which provides a physics-based compact representation of bipolar resistive random access memory (RRAM) structures incorporating metal-oxide layers. Being an extended memristor formulation, this framework successfully describes the nonlinear resistive switching dynamics observed across a diverse range of device implementations, particularly capturing critical phenomena such as apparent threshold behavior, temporal voltage-dependent switching characteristics, and multilevel conductance programming capabilities. Flux-charge formulations have further extended the modeling landscape to phase-change memory devices [9], while voltage-controlled threshold models have been developed specifically for synaptic circuit design applications [10]. Alternative modeling approaches have been developed to capture memristive behavior through different mathematical frameworks. The data-driven stochas-

tic model [11] addresses programming uncertainty by incorporating real-time randomness into resistance evolution, determined through extensive characterization of data using the data-driven methodology. Efficient implementations have demonstrated voltage-based state control approaches that capture nonlinear characteristics while maintaining flexibility across different device types [12]. For integration into circuit simulation environments, modified nodal analysis (MNA) formulations [13] provide automated and flexible modeling techniques that preserve the advantages of traditional circuit analysis methods, including sparsity and generality across various simulation types. Approximated linear models implemented in Verilog-A have also been proposed to reduce computational cost while preserving the ability to capture parasitic effects in crossbar memory arrays [14]. Parameter extraction methodologies [15] have further advanced model optimization by enabling automatic fitting to device characterization data, thereby streamlining the modeling workflow for experimenters working with tantalum oxide and other memristive devices.

The Generalized Metastable Switch (GMS) model [16] provides enhanced accuracy in modeling Self-Directed Channel (SDC) memristors through a semi-empirical formulation where state variable dynamics depend on transition probabilities between metastable states. However, such symbolic models exhibit limited generalization capability when driving signal parameters vary [17], posing challenges for applications with highly dynamic operating conditions such as memristor-based chaotic oscillators [18]. Memristive elements are also widely employed in chaotic systems facing similar modeling challenges, including Hopfield neural networks designed to generate multi-scroll attractors through offset boosting techniques [19]. To address these limitations, Minati et al. [20] introduced the Mean Metastable Switch (MMS) model, which simplifies the GMS framework by computing the derivative of the internal state variable through linear interpolation of mean values obtained from Gaussian sampling of each probabilistic function, while eliminating the Schottky effect to preserve model simplicity. Subsequently, this approach was extended in the Generalized Mean Metastable Switch (GMMS) model [21], [22], which reincorporates the Schottky diode effect originally described in the GMS formulation.

Numerical simulation of memristors with bounded state variables presents additional computational challenges. Window functions [2], [3] employed to constrain internal state variables within physically admissible limits introduce discontinuities in the governing differential equations, which can generate numerical artifacts when standard integration methods are applied [23]. Boundary condition-based model for studying memristor nanostructures was proposed in [24]. A Poincaré map approach [25] partitions the state space into smooth regions to enable accurate trajectory computation across discontinuity boundaries. Furthermore, while physics-based models offer high accuracy, their mathematical complexity hinders large-scale circuit simulation, motivating efforts toward explicit reformulations [26]. Recently, Hajtó et al. [27] proposed a physics-inspired state characterisation approach for SDC memristors, extending the Generalised MSS conduction model by introducing state-dependent Schottky diode components and a noise-aware minimum-variance state estimation protocol from voltage-current pairs. Their work addresses the complementary problem of *static* state identification at a given instant, demonstrating that accurate characterisation of the nonlinear V-I relationship requires going beyond simple Ohmic assumptions.

In [28], the authors attempted to model memristive behavior using the Physics-Informed Neural Networks (PINNs) framework. Although the reported results indicate the potential of this method, the study is not grounded in experimental measurements of physical memristor devices. Instead, the analysis is limited to comparisons with the outcomes of existing simulation models. Moreover, the training data employed during the neural network learning process were generated based on previously developed theoretical models, thereby limiting the ability to assess the method's accuracy in the context of real-world physical systems.

In [29], the authors introduce the concept of deep neural models in which the dynamics of the hidden state of a dynamical system is governed by an ordinary differential equation (ODE). The training process is performed in an end-to-end manner, meaning that all parameters are optimized simultaneously within a single training procedure. A key innovation of the proposed approach is a novel backpropagation technique, which relies on solving the corresponding adjoint ODE backward in time using adjoint sensitivity methods. This formulation enables efficient gradient computation and facilitates the application of neural ODEs in various architectures, including continuous-depth residual networks and generative flow-based models. This paper investigates the application of artificial neural networks to the modeling and characterization of memristive devices, with particular emphasis on data-driven approaches that leverage experimental measurements from physical systems. While numerous physics-based analytical models have been developed to capture memristor dynamics, these approaches often require extensive parameter tuning and may exhibit limited accuracy when device behavior deviates from idealized assumptions. To address these limitations, we propose and evaluate two Neural Ordinary Differential Equation (Neural ODE) architectures for memristor modeling. The first, SingleNN-MemODE, employs a physics-informed memductance formulation with a neural network modeling only the state dynamics. The second, DualNN-MemODE, uses neural networks for both memductance and state dynamics, enabling a higher-dimensional latent representation. Both proposed approaches are applied to modeling Self-Directed Channel (SDC) memristors and trained directly on experimental data from physical devices.

The primary objective of this study is threefold: first, to assess whether neural network-based models can accurately reproduce the dynamic I-V characteristics and internal state evolution of SDC memristors across diverse operating conditions; second, to evaluate the generalization capability of

trained neural models when applied to input signals outside the training distribution; and third, to provide a systematic comparison with established theoretical models in terms of prediction accuracy, computational efficiency, and practical applicability to circuit simulation scenarios. Through comprehensive experimental validation using measurements from physical devices, we demonstrate that both Neural ODE models outperform the MMS and GMMS benchmarks in prediction accuracy while maintaining comparable or shorter training times, highlighting their potential as complementary tools to traditional physics-based modeling approaches in the design and analysis of memristive circuits and neuromorphic computing systems.

The remaining part of this paper is organized as follows. In Sec. II, the neural network architecture is presented and two neural network-based models (SingleNN-MemODE and DualNN-MemODE) are proposed. The objective function defining the optimization problem to be solved is defined and existing memristors models used for comparison are briefly described. In Sec. III, proposed models are trained using experimental data to obtain accurate models of real SDC memristors. The results are compared with the results obtained using existing methods. The influence of hyperparameters of the proposed methods on their performance is discussed. The last section concludes the study.

II. MODELING MEMRISTORS USING NEURAL NETWORKS

This study utilizes experimental data obtained from Self-Directed Channel (SDC) memristors doped with tungsten. The structure and properties of these devices have been described in detail in the literature, e.g., in [30], [31].

This study utilizes experimental data obtained from commercially available tungsten-doped Self-Directed Channel (SDC) memristors manufactured by Knowm Inc. These are two-terminal thin-film devices based on a chalcogenide glass active layer with the following layer structure (from bottom to top): W (bottom electrode), Ge_2Se_3 (adhesion layer), W+Ge_2Se_3 (active layer with injection doping), Ge_2Se_3 (intermix layer), SnSe (assist layer), Ge_2Se_3 (intermix layer), Ag (ion source layer), Ge_2Se_3 (adhesion layer), and W (top electrode). Resistive switching is governed by the migration of Ag^+ ions from the source layer into and out of the active Ge_2Se_3 region, where they form and disrupt conductive filaments that modulate the device conductance between the high-resistance state (HRS) and low-resistance state (LRS). The tungsten doping in the active layer modifies the filament formation dynamics and enhances switching stability. Detailed descriptions of the device structure, fabrication process, and material characterisation are provided in [27], [30], [31].

The experimental setup, illustrated in Fig. 1, consists of an SDC memristor connected in series with a resistor. The sinusoidal input voltage is supplied by an arbitrary waveform generator. Voltage signals are measured using a data acquisition system (myDAQ University Kit). The series resistance is $R_s = 5.11\ \mathrm{k\Omega}$.

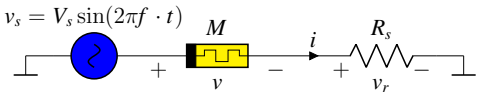

FIGURE 1. Schematic diagram of the measurement setup used for the SDC memristor. The device is connected in series with a resistor, and the input voltage is applied via an arbitrary waveform generator.

Measurements are carried out for the following combinations of supply voltage amplitudes and frequencies: $V_s \in \{0.5,\ 1.0,\ 1.5\}$ [V], $f \in \{1,\ 5,\ 10,\ 20,\ 50,\ 100\}$ [Hz]. All experimental data are obtained from a single physical SDC memristor device. For each of the 18 combinations of supply voltage amplitude and frequency, 100 consecutive periods of the excitation signal are recorded, yielding 10^5 samples per trajectory (1000 samples per period). No repeated measurements under identical conditions from separate sessions are performed; instead, the 100-period recordings capture both the initial transient response and the steady-state behaviour within a single continuous acquisition, providing sufficient statistical content for training and validation.

Sinusoidal excitation is chosen for device characterisation for several reasons. First, it provides continuous, symmetric cycling through both the SET (positive half-cycle) and RESET (negative half-cycle) transitions within each period, enabling observation of the complete pinched hysteresis loop—the defining signature of memristive behaviour. Second, the single-frequency content of a sinusoidal signal allows for a clear separation of the device response across different frequency regimes, enabling systematic investigation of frequency-dependent phenomena such as hysteresis loop area reduction and the transition from quasi-static to rate-limited switching behaviour. Third, the smooth and continuously differentiable nature of the sinusoidal signal is well suited for training Neural ODE models, which rely on solvers that benefit from the absence of discontinuities in the driving signal. Fourth, sinusoidal characterisation is a widely adopted standard in the memristor modelling literature [4], [21], [32], facilitating direct comparison of our results with those obtained under equivalent experimental conditions.

The analyzed model of a memristor follows the general theoretical framework of memristive devices originally introduced in [33], and is expressed as

$$i(t) = \mathcal{G}\left(\mathbf{x}(t), v(t)\right) v(t), \tag{1a}$$

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = f\left(\mathbf{x}(t), v(t)\right), \tag{1b}$$

where $\mathbf{x}$ denotes the vector of internal state variables, whose temporal evolution is governed by the function f, and $\mathcal{G}$ represents the memductance of the device. The fundamental challenge in memristor modeling lies in accurately identifying the functional forms of $\mathcal{G}(\mathbf{x}, v)$ and $f(\mathbf{x}, v)$.

A. OBJECTIVE FUNCTION

The objective function $\mathcal{L}$ to be minimized in the optimization process is based on trajectories generated by the dynamical

system. It is designed to simultaneously account for both the signal values and its dynamic structure. To this end, it is defined as the sum of four components, each serving a distinct and complementary role in the optimization process

$$\mathcal{L} = \lambda_{\mathrm{b}}\mathcal{L}_{\mathrm{b}} + \lambda_{\mathrm{v}}\mathcal{L}_{\mathrm{v}} + \lambda_{\mathrm{c}}\mathcal{L}_{\mathrm{c}} + \sum_{i=1}^{N_c} \lambda_{\mathrm{con},i}\mathcal{L}_{\mathrm{con},i}(x). \quad (2)$$

The first term $\mathcal{L}_{\mathrm{b}}$ denotes the primary error computed as the normalized mean squared error (NMSE) between predicted and reference values. $\mathcal{L}_{\mathrm{v}}$ enforces accurate modeling of derivatives of state variables, $\mathcal{L}_{\mathrm{c}}$ promotes consistency of the trajectories in terms of their curvature, and $\mathcal{L}_{\mathrm{con},i}(x)$ enforce fulfillment of physical or structural constraints that the model is required to satisfy. The weighting coefficients λ_{b}, λ_{v}, and λ_{c} balance the influence of each term according to the priorities of the modeling task. Their values are calculated during each optimization step using the the adaptive loss balancing procedure described in Algorithm 1 [34], [35].

Algorithm 1 Adaptive Loss Balancing with Clamping

Require: Loss terms $\mathcal{L}_{\mathrm{b}}, \mathcal{L}_{\mathrm{v}}, \mathcal{L}_{\mathrm{c}}, \varepsilon > 0$, bounds $[\lambda_{\min}, \lambda_{\max}]$
Ensure: Adaptive weights $\lambda_{\mathrm{b}}, \lambda_{\mathrm{v}}, \lambda_{\mathrm{c}}$

1: Compute the geometric mean:
$\overline{\mathcal{L}} \leftarrow \exp\left(\frac{1}{3}\left(\ln(\mathcal{L}_{\mathrm{b}}+\varepsilon) + \ln(\mathcal{L}_{\mathrm{v}}+\varepsilon) + \ln(\mathcal{L}_{\mathrm{c}}+\varepsilon)\right)\right)$
2: $\lambda_{\mathrm{b}} \leftarrow \mathrm{clamp}(\frac{\overline{\mathcal{L}}}{\mathcal{L}_{\mathrm{b}}+\varepsilon}, \lambda_{\min}, \lambda_{\max})$
3: $\lambda_{\mathrm{v}} \leftarrow \mathrm{clamp}(\frac{\overline{\mathcal{L}}}{\mathcal{L}_{\mathrm{v}}+\varepsilon}, \lambda_{\min}, \lambda_{\max})$
4: $\lambda_{\mathrm{c}} \leftarrow \mathrm{clamp}(\frac{\overline{\mathcal{L}}}{\mathcal{L}_{\mathrm{c}}+\varepsilon}, \lambda_{\min}, \lambda_{\max})$
5: **return** $(\lambda_{\mathrm{b}}, \lambda_{\mathrm{v}}, \lambda_{\mathrm{c}})$

Let us consider two sequences $z_k = (z_{k,n})_{n=1}^{N}$ and $\tilde{z}_k = (\tilde{z}_{k,n})_{n=1}^{N}$, where $z \in \{v, i, \dot{v}, \dot{i}, \ddot{v}, \ddot{i}\}$ is a selected variable (voltage or current) or its derivative obtained from measurements, $\tilde{z}$ is the predicted value of z, N is the number of samples and $k = 1, 2, \ldots, N_{\mathrm{traj}}$ is the sequence index. The sequences are normalized utilizing the trajectory-specific standard deviation $\sigma_{z,k}$ to ensure statistical consistency within each dataset

$$\sigma_{z,k} = \sqrt{\frac{1}{N-1}\sum_{n=1}^{N}(z_{k,n} - \bar{z}_k)^2}, \quad (3)$$

where the temporal mean $\bar{z}_k$ is defined as $\bar{z}_k = \frac{1}{N}\sum_{n=1}^{N} z_{k,n}$.

By implementing trajectory-specific normalization rather than global standardization, the methodology preserves the inherent temporal dynamics and statistical properties unique to each circuit configuration while simultaneously ensuring that amplitude disparities do not introduce systematic bias during modeling procedures.

The normalized mean squared error between sequences $(z_{k,n})_{n=1}^{N}$ and $(\tilde{z}_{k,n})_{n=1}^{N}$ is defined as

$$\mathrm{NMSE}(z_k, \tilde{z}_k) = \frac{\mathrm{MSE}\left(z_k, \tilde{z}_k\right)}{\sigma_{z,k}^2} = \frac{1}{N}\sum_{n=1}^{N}\left(\frac{z_{k,n}}{\sigma_{z,k}} - \frac{\tilde{z}_{k,n}}{\sigma_{z,k}}\right)^2.$$

The choice of the normalized mean squared error (NMSE) as the primary metric is motivated by the substantial disparity between measured amplitudes of voltage and current signals, which differ by up to three orders of magnitude.

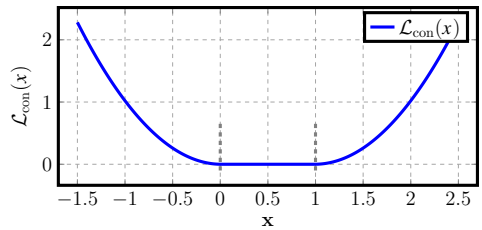

FIGURE 2. The constraint function $\mathcal{L}_{\mathrm{con}}(x)$ for $a = 0$, $b = 1$, $\gamma = 0.0025$.

Using raw MSE as the optimization objective would lead to models that accurately reproduce voltage waveforms while exhibiting poor current prediction, which is the quantity most relevant for characterizing memristive switching behavior.

The primary component of the loss function is responsible for model fitting to experimental voltage-current data:

$$\mathcal{L}_{\mathrm{b}} = \frac{1}{N_{\mathrm{traj}}}\sum_{k=1}^{N_{\mathrm{traj}}}\left(\mathrm{NMSE}(v_k, \tilde{v}_k) + \mathrm{NMSE}(i_k, \tilde{i}_k)\right). \quad (4)$$

The first-order dynamics comparison $\mathcal{L}_{\mathrm{v}}$ enables evaluation of temporal derivative agreement between model predictions and experimental data, particularly critical for capturing rapid memristor state transitions

$$\mathcal{L}_{\mathrm{v}} = \frac{1}{N_{\mathrm{traj}}}\sum_{k=1}^{N_{\mathrm{traj}}}\left(\mathrm{NMSE}(\dot{v}_k, \tilde{\dot{v}}_k) + \mathrm{NMSE}(\dot{i}_k, \tilde{\dot{i}}_k)\right). \quad (5)$$

The second-order comparison term

$$\mathcal{L}_{\mathrm{c}} = \frac{1}{N_{\mathrm{traj}}}\sum_{k=1}^{N_{\mathrm{traj}}}\left(\mathrm{NMSE}(\ddot{v}_k, \tilde{\ddot{v}}_k) + \mathrm{NMSE}(\ddot{i}_k, \tilde{\ddot{i}}_k)\right) \quad (6)$$

facilitates superior representation of curvature properties.

This term is particularly important for modeling systems exhibiting complex, highly dynamic responses.

To enforce that a parameter x stays within the bounds $x \in [a, b]$, the smooth constraint loss function $\mathcal{L}_{\mathrm{con}}$ is used

$$\mathcal{L}_{\mathrm{con}}(x) = \mathbb{E}\left[\sigma\left(\frac{a-x}{\gamma}\right)(a-x)^2 + \sigma\left(\frac{x-b}{\gamma}\right)(x-b)^2\right], \quad (7)$$

where $\sigma(z) = (1 + e^{-z})^{-1}$ is the sigmoid function,

γ is a scaling factor controlling the steepness of the transition near the bounds, and $\mathbb{E}[\cdot]$ denotes the expected value (ensemble average over samples or trajectories). The first term in (7) penalizes values $x < a$, while the second term penalizes values $x > b$. Using the smooth constraint loss function is necessary to maintain differentiability for gradient-based optimization algorithms.

The function $\mathcal{L}_{\mathrm{con}}(x)$ for the parameter values: $a = 0$, $b = 1$, $\gamma = 0.0025$ over the domain $x \in (-0.5, 1.5)$ is plotted in Fig. 2.

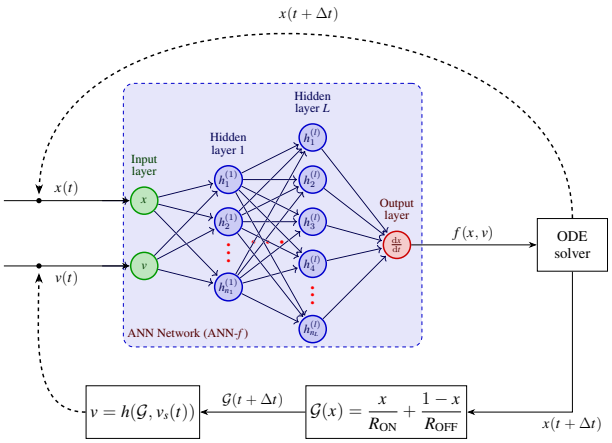

FIGURE 3. The SingleNN-MemODE approach. The artificial neural network (ANN) evaluates the function $f(x, v)$, which is then integrated by the ODE solver to predict the state at the next time step. The upper dashed feedback loop indicates the recurrent nature of the process, where the output state is fed back as input for subsequent predictions. The memductance $\mathcal{G}(x)$ is defined using a deterministic function based on the internal variable x. The function $h(\mathcal{G}, v_s(t))$ computes the memristor voltage v given the memductance $\mathcal{G}$ and supply voltage $v_s(t)$.

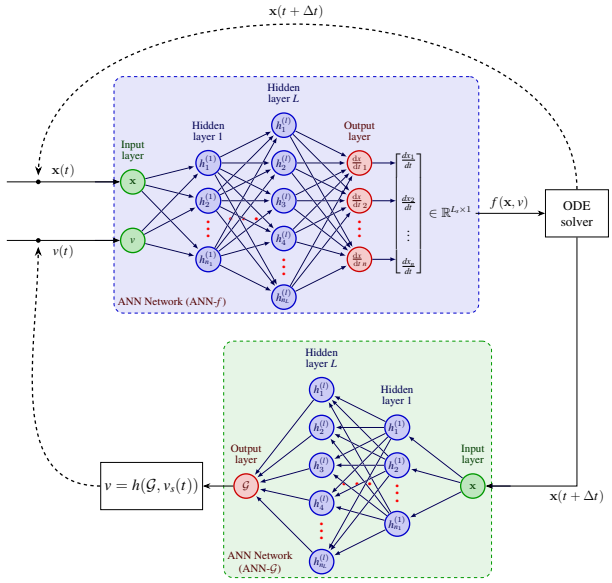

FIGURE 4. The DualNN-MemODE approach. The artificial neural network (ANN) evaluates the function $f(\mathbf{x}, v)$, which is then integrated by the ODE solver to predict the state at the next time step. The second ANN models the memductance $\mathcal{G}(\mathbf{x})$ based on $\mathbf{x}(t+\acute{t})$. The function $h(\mathcal{G}, v_s(t))$ computes the memristor voltage v given $\mathcal{G}$ and $v_s(t)$.

B. NEURAL ODE BASED MEMRISTOR MODELING

In this study, two neural network-based modeling approaches are proposed. The first one, denoted as **SingleNN-MemODE**, employs a deterministic formulation of the memristor memductance $\mathcal{G}$, while the function f defining the dynamics of internal variables is modeled using a neural network (see Fig. 3). In the second one, referred to as **DualNN-MemODE**, both $\mathcal{G}$ and f are modeled using neural networks (see Fig. 4).

For the SingleNN-MemODE approach, the function f is implemented using an artificial neural network with a single output. For the DualNN-MemODE approach the number of outputs is equal to the number of internal variables and may be larger than 1. For both cases, the neural network consists of interconnected nodes (neurons) that process information through weighted connections. Each neuron in the network computes its output according to:

$$y_k = \sigma\left(\sum_{i=1}^{M_k} w_{k,i}x_i + b_k\right), \tag{8}$$

where k is the neuron's index, M_k is the number of inputs for the kth neuron, $w_{k,i}$ are the weights that determine the strengths of input connections x_i of the kth neuron, b_k is the bias term that provides an adjustable threshold for neuron activation, and σ is the activation function (e.g., sigmoid, ReLU, or tanh) that introduces nonlinearity into the model.

In the SingleNN-MemODE approach, the memductance is defined on the basis of the physical mechanisms underlying self-directed channel (SDC) devices. In such memristors, the resistance evolves as a result of Ag^+ ions migration and the formation of conductive filaments. Consequently, the device memductance exhibits a continuous transition between different resistance states. This transition can be described as a weighted combination of the limiting resistance values, modulated by the internal variable x

$$\mathcal{G}(x) = \frac{x}{R_{ON}} + \frac{1-x}{R_{OFF}}, \tag{9}$$

where R_{ON} and R_{OFF} denote the resistances in the low-resistance and high-resistance states, respectively. During the optimization process, the network parameters (weights $w_{k,i}$ and biases b_k) are tuned jointly with the values of R_{ON} and R_{OFF}. For this approach the following constraints are applied:

- $R_{ON} \in (500\,\Omega, R_{OFF})$,
- $R_{OFF} \in (R_{ON}, 200\,\mathrm{k}\Omega)$,
- $x(t) \in (0, 1)$.

The lower bound for R_{ON} and the upper bound for R_{OFF} are based on empirical measurements.

In the DualNN-MemODE approach, the memductance is modeled using the second neural network. This neural network takes the current state $\mathbf{x}$ as the input and produces the corresponding memductance $\mathcal{G}(\mathbf{x})$ as the output. The memristor state $\mathbf{x}$ is not constrained to be one-dimensional, enabling a more flexible representation of the device's internal dynamics. The state variables are not restricted to any specific range, and hence no explicit constraints are applied to them. For this approach the only constraint is

- $\mathcal{G}(\mathbf{x}) \in (5\,\mu S, 2\,\mathrm{m}S)$.

The learning process involves iteratively adjusting the network parameters to minimize the loss function that quantifies the difference between predicted and target outputs. This is achieved through backpropagation algorithm combined with the gradient descent optimization, where gradients of the loss

function with respect to each parameter are computed and used to update the weights and biases in the direction that reduces the overall error.

C. NEURAL NETWORK ARCHITECTURE

The neural network training is implemented using the `JAX` library [36] for array-oriented numerical computation and `Equinox` [37] for neural network construction in `Python`. The parameter gradients of the ODE solution are computed using adjoint sensitivity methods [29], [38]. The `tsit5` Runge-Kutta solver with adaptive step-size control [39] is employed for its computational efficiency and numerical accuracy.

The neural network architecture, illustrated in Fig. 5, employs an expansion–compression (diamond-shaped) topology that incorporates a double reduction in dimensionality in both the initial and final hidden layers. This architectural configuration substantially decreases the overall number of trainable parameters and helps to mitigate vanishing and exploding gradient issues commonly encountered in deep architectures processing temporal sequences.

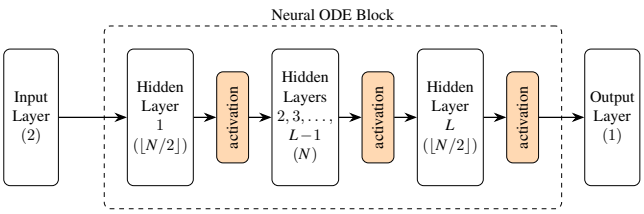

FIGURE 5. The Neural ODE block for the computation of $f(x(t), v(t))$. The network consists of an input layer receiving two inputs, multiple hidden layers with decreasing-increasing-decreasing neuron counts ($\lfloor N/2 \rfloor, N, \ldots, N, \lfloor N/2 \rfloor$), activation functions between layers, and a single output.

Several techniques are tested for the optimization of network weights and the `Adam` (Adaptive Moment Estimation) optimizer is selected due to its superior performance. The choice of an optimizer and its parameters is guided by hyperparameter tuning conducted with the `Optuna` framework [40]. The hyperparameter search space comprises the learning rate, weight decay, batch size, and architecture parameters like the width and the depth of the neural network. The optimal configuration is selected based on minimizing the objective function on the validation set, ensuring that the model generalizes well to unseen data.

D. EXISTING MEMRISTOR MODELS FOR COMPARISON

To evaluate the effectiveness of the proposed models, their performance is compared with the performance of two existing memristor models: the Mean Metastable Switch (MMS) model [20] and the Generalized Mean Metastable Switch Memristor (GMMS) model [16], [21].

The MMS model is a simplified representation of memristor's behavior, focusing on the average switching characteristics rather than the detailed dynamics. It captures essential features of the device by describing transitions between the high-resistance state (HRS) and the low-resistance state (LRS) based on the applied voltage. This model describes memristor's dynamics as

$$\frac{\mathrm{d}x}{\mathrm{d}t} = \frac{1}{\tau}\left(f_{ON}(v)(1-x) - f_{OFF}(v)x\right), \tag{10}$$

$$f_{ON}(v) = \sigma\left(-\beta(v - V_{ON})\right), \tag{11}$$

$$f_{OFF}(v) = 1 - \sigma\left(\beta(v - V_{OFF})\right), \tag{12}$$

where $\sigma(z) = (1 + e^{-z})^{-1}$, $\beta = \frac{q}{kT}$, k is the Boltzmann constant, T is the absolute temperature, q represents the elementary charge, while V_{ON} and V_{OFF} denote the switching voltages to LRS and HRS states, respectively. The instantaneous memductance is defined in (9).

In the GMMS model memristors are represented as a parallel connection of memory-dependent element and a Schottky diode [16], [21]. This formulation extends the original metastable switch concept by incorporating nonlinear current–voltage characteristics, enabling a more accurate representation of memristor's behavior across a wide range of operating conditions. The evolution of the state variable is governed by (10), while the current–voltage relationship is

$$\begin{aligned} i(t) &= \eta i_{\mathrm{md}}(v,x) + (1-\eta) i_{\mathrm{d}}(v), \\ i_{\mathrm{d}}(v) &= \alpha_{\mathrm{f}} e^{\beta_{\mathrm{f}} v} - \alpha_{\mathrm{r}} e^{-\beta_{\mathrm{r}} v}, \\ i_{\mathrm{md}}(v,x) &= \mathcal{G}(x) v, \end{aligned} \tag{13}$$

where $\eta \in [0, 1]$ is the parameter representing the ratio of the memory dependent current i_{md} and total current i flowing through the device. The parameters α_f, β_f, α_r, and β_r are positive constants characterizing the forward and reverse current across the Schottky barrier.

The inclusion of additional parameters and nonlinearities in the GMMS model increases model's complexity and poses challenges for parameter optimization. An additional challenge arises due to the Schottky effect. When the memristor is connected in series with a resistor, this requires solving a set of differential-algebraic equations (DAEs) of the form

$$\begin{cases} v_{\mathrm{s}} = v_{\mathrm{r}} + v, \\ \dfrac{\mathrm{d}x}{\mathrm{d}t} = \dfrac{1}{\tau}\left(f_{ON}(v)(1-x) - f_{OFF}(v)x\right), \end{cases} \tag{14}$$

where v_{s} denotes the supply voltage, $v_{\mathrm{r}} = R_{\mathrm{s}} i$ is the voltage across the resistor, and v is the voltage across the memristor.

Parameter estimation for existing memristor models employs a two-stage hybrid optimization strategy. Global exploration is performed using Adaptive Differential Evolution with radius-limited sampling from the `BlackBoxOptim` package [41] in `Julia`, followed by local refinement via the L-BFGS-B quasi-Newton method [42] implemented in `Optim.jl`. This approach efficiently identifies optimal parameter sets while avoiding premature convergence to local minima.

III. RESULTS

During the hyperparameter tuning, a range of neural network architectures is systematically evaluated in order to identify the most suitable configuration for modeling memristor

dynamics. The evaluation considers the number of hidden layers, the number of neurons per layer, and optimization settings (e.g. the batch size and the learning rate), with the aim of achieving a balance between model accuracy and generalization capabilities, as described in Sec. II-C. Among these settings, the batch size determines the number of training samples processed before the model parameters are updated, thereby influencing both the convergence speed and the stability of the learning process. In this study, the batch size corresponds to the number of trajectories processed in parallel before each parameter update. The learning rate, on the other hand, controls the step size of each parameter update during training. An excessively high learning rate may cause the optimization process to diverge, while a low value may result in slow convergence or being trapped in a local minimum.

In this study, the dataset consists of 18 trajectories obtained from measurements for all combinations of supply voltage amplitudes $V_\mathrm{s} \in \{0.5, 1.0, 1.5\}$ V and frequencies $f \in \{1, 5, 10, 20, 50, 100\}$ Hz. The number of samples for each trajectory is 10^5 (100 periods of the supply voltage per trajectory, 1000 samples per period) as mentioned in Section II. This dataset is divided into the training dataset consisting of 14 trajectories and the validation datasets comprising 4 trajectories. Trajectories used for training and validation are selected to cover a diverse range of memristor's operating conditions, thereby enhancing the model's ability to generalize across different dynamic regimes. The validation dataset contains the following pairs of supply voltages and frequencies: $(1.5\,\mathrm{V}, 5\,\mathrm{Hz})$, $(1\,\mathrm{V}, 1\,\mathrm{Hz})$, $(1\,\mathrm{V}, 100\,\mathrm{Hz})$, and $(0.5\,\mathrm{V}, 5\,\mathrm{Hz})$. The selection of validation trajectories follows a deliberate strategy: the held-out pairs span the corners and interior of the (V_s, f) operating grid, ensuring that generalization is evaluated across dynamically distinct regimes rather than within interpolated conditions. This approach is particularly appropriate for physical dynamical systems, where each trajectory corresponds to a qualitatively different operating regime characterized by distinct switching dynamics, hysteresis loop morphology, and frequency-dependent relaxation behavior. In such settings, standard (k)-fold cross-validation over randomly partitioned trajectories is less informative, as it primarily assesses interpolation capability within similar regimes rather than the model's ability to extrapolate across the operating parameter space [43].

During the hyperparameter optimization stage, the optimal configuration of the SingleNN-MemODE architecture found contains four hidden layers with 64, 128, 128, and 64 neurons in consecutive layers. The model is trained using the `AdamW` optimizer with a learning rate of approximately $1.89 \cdot 10^{-2}$ and the batch size of 2. The swish activation function $\varphi(x) = x\sigma(x)$ is employed for the hidden layers, while the final layer utilizes the linear activation function $\varphi(x) = x$.

For the DualNN-MemODE architecture, the optimal neural network configuration responsible for computing $f(\mathbf{x}, v)$ consists of four hidden layers with 120, 240, 240, and 120 neurons in consecutive layers and with 6 latent states, corresponding to the dimensionality of the internal state vector $\mathbf{x}$. The auxiliary neural network responsible for computing $\mathcal{G}$ includes one hidden layer with 80 neurons. Both networks are trained using the `AdaBelief` optimizer with a learning rate of $6.3 \cdot 10^{-3}$ and the batch size of 14. The first neural network employs the GELU activation function $\varphi(x) = x \cdot \Phi(x) \approx 0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$ in the hidden layers and the linear activation in the output layer, whereas the second network uses the ReLU activation function $\varphi(x) = \max(0, x)$ with the sigmoid output activation.

A representative learning curve corresponding to the SingleNN-MemODE architecture, is presented in Fig. 6. The training process is conducted over 360 epochs, with the objective function $\mathcal{L}$ monitored on both the training and validation datasets to provide insights into convergence behavior and potential overfitting. The learning curve demonstrates a consistent decrease in values of $\mathcal{L}$ for both datasets, which indicates effective optimization and a satisfactory model fitting to the experimental data. The close agreement between the values of the objective functions for the training and validation datasets further confirms the generalization ability of the selected architecture.

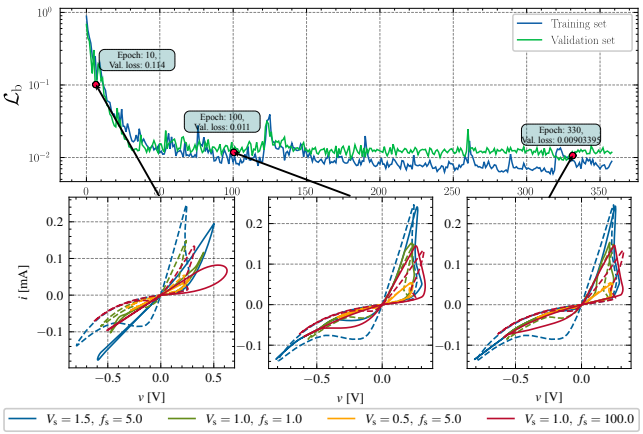

FIGURE 6. Learning curve for the SingleNN-MemODE neural network architecture during training over 360 epochs. The insets illustrate *v–i* hysteresis loops at selected epochs, where the dashed lines correspond to the measured responses and the solid lines denote the model predictions.

Fig. 6 also presents the *v–i* hysteresis characteristics obtained at selected stages of the optimization process. These snapshots highlight how the model progressively refines its internal representation of the device dynamics during training. The gradual improvement of the hysteresis fit across epochs illustrates the capability of the neural network to capture the nonlinear and history-dependent behavior of the memristor, thereby validating the suitability of the chosen architecture and training configuration.

The effectiveness of optimizing the full loss function $\mathcal{L}$ defined in (2) was compared against optimizing only the base loss function $\mathcal{L}_\mathrm{b}$, defined in (4). For instance, when optimizing only the base loss function for the SingleNN-MemODE architecture, the final value obtained for the validation dataset is $\mathcal{L}_\mathrm{b} = 2.24{\times}10^{-2}$, whereas optimizing the full loss function yields a lower value of $\mathcal{L}_\mathrm{b} = 1.85{\times}10^{-2}$. This result indicates

that incorporating additional terms $\mathcal{L}_v$ and $\mathcal{L}_c$ improves convergence and enhances the overall predictive performance of the model.

This outcome also indicates that the derivative-based loss terms do not overfit to measurement noise: if $\mathcal{L}_v$ and $\mathcal{L}_c$ were fitting noise artifacts rather than genuine dynamic structure, the validation loss would increase rather than decrease when these terms are included.

Representative simulation results demonstrating the neural network model's predictive capabilities are presented in Fig. 7 through comparative analysis of a tungsten-doped memristor device subjected to sinusoidal voltage excitation with an amplitude of 1.5 V and frequency of 100 Hz.

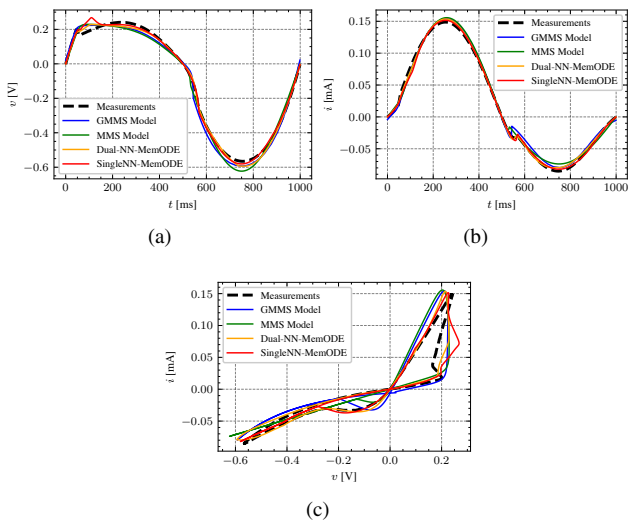

FIGURE 7. Simulation test results for tungsten-doped memristor dynamics under sinusoidal voltage excitation (amplitude: 1.0 V, frequency: 1.0 Hz); (a) memristor's voltage waveform, (b) memristor's current response, (c) voltage-current hysteresis loop.

The analysis encompasses the voltage waveform, the current waveform and the voltage-current hysteresis relationships. The voltage profile confirms the fidelity of the input stimulus. The temporal current response reveals the device's instantaneous electrical behavior and switching kinetics. The voltage-current hysteresis loops ($v - i$) illustrate the fundamental memory properties that define memristive behavior.

The hysteresis loops are particularly diagnostic of memristor performance, as their shape, area, and switching thresholds directly reflect the underlying ionic transport mechanisms and structural modifications responsible for resistive switching. The pinched hysteresis loop, which intersects at the origin, serves as a definitive signature of memristive behavior, while the loop area quantifies the energy dissipation associated with switching events. These features validate the ability of the neural network model to capture both the static and dynamic aspects of memristor operation.

TABLE 1. Comparative analysis of the loss function $\mathcal{L}_b$ for different supply voltage amplitudes V_s and frequencies f_s. Row-wise minimum and maximum loss values are written in green and red, respectively. Records belonging to the validation dataset are marked by bounding boxes.

Loss function values for $V_s = 0.5$ V				
f_s [Hz]	SingleNN-MemODE	DualNN-MemODE	GMMS Model	MMS Model
1	9.31×10^{-3}	1.22×10^{-3}	3.94×10^{-2}	2.43×10^{-2}
5	3.40×10^{-3}	6.43×10^{-3}	2.97×10^{-2}	1.46×10^{-2}
10	3.05×10^{-3}	2.13×10^{-3}	2.85×10^{-2}	2.60×10^{-2}
20	3.79×10^{-3}	1.02×10^{-3}	3.25×10^{-2}	1.14×10^{-1}
50	2.03×10^{-2}	5.60×10^{-3}	3.68×10^{-2}	3.45×10^{-2}
100	2.82×10^{-2}	1.07×10^{-2}	9.91×10^{-2}	1.11×10^{-1}
Loss function values for $V_s = 1$ V				
f_s [Hz]	SingleNN-MemODE	DualNN-MemODE	GMMS Model	MMS Model
1	5.28×10^{-3}	4.47×10^{-3}	1.30×10^{-2}	1.63×10^{-2}
5	8.25×10^{-3}	2.67×10^{-3}	1.46×10^{-2}	1.71×10^{-2}
10	4.72×10^{-3}	8.86×10^{-4}	1.49×10^{-2}	1.58×10^{-2}
20	7.77×10^{-3}	5.53×10^{-3}	1.88×10^{-2}	1.90×10^{-2}
50	3.29×10^{-2}	7.37×10^{-3}	6.29×10^{-2}	6.22×10^{-2}
100	4.48×10^{-2}	8.58×10^{-3}	4.53×10^{-2}	3.25×10^{-2}
Loss function values for $V_s = 1.5$ V				
f_s [Hz]	SingleNN-MemODE	DualNN-MemODE	GMMS Model	MMS Model
1	1.74×10^{-2}	2.48×10^{-3}	1.75×10^{-2}	4.74×10^{-2}
5	1.57×10^{-2}	3.60×10^{-3}	2.15×10^{-2}	3.71×10^{-2}
10	1.55×10^{-2}	3.70×10^{-3}	2.08×10^{-2}	2.31×10^{-2}
20	1.32×10^{-2}	7.93×10^{-3}	2.07×10^{-2}	2.41×10^{-2}
50	1.37×10^{-2}	8.41×10^{-3}	1.69×10^{-2}	4.70×10^{-2}
100	2.64×10^{-2}	1.20×10^{-2}	5.54×10^{-2}	2.41×10^{-2}

A. COMPARATIVE ANALYSIS WITH THE MMS AND GMMS MODELS

To assess the performance of the neural ODE approach, a comprehensive comparative analysis is conducted between the developed neural network models and the existing models of SDC memristors: the Mean Metastable Switch (MMS) model and the Generalized Mean Metastable Switch (GMMS) model. This comparison is particularly significant as the MMS and GMMS models have been extensively validated against experimental data and serve as benchmarks for memristor circuit simulation [21], [32].

Table 1 presents a comparative analysis of the loss functions $\mathcal{L}_b$ obtained for the proposed and existing models, offering a quantitative assessment of their performance. The highest loss value in each row is marked in red, whereas the lowest is marked in green. Overall, the neural network model consistently achieves substantially lower loss values, demonstrating superior accuracy and fidelity in representing the device behavior. The results highlight the neural network's enhanced capability to reproduce complex aspects of memristor dynamics that are insufficiently captured by the phenomenological MMS/GMMS framework.

Table 2 reports the average loss function values $\mathcal{L}_b$ calculated for the validation, training and the whole datasets for the proposed and existing models. Results obtained for

TABLE 2. Average values of the loss function $\mathcal{L}_b$ for the validation, training and the whole datasets.

	SingleNN-MemODE	DualNN-MemODE	GMMS Model	MMS Model
Validation dataset				
$\mathcal{L}_b$	1.73×10^{-2}	5.77×10^{-3}	2.74×10^{-2}	2.51×10^{-2}
Training dataset				
$\mathcal{L}_b$	1.46×10^{-2}	5.11×10^{-3}	3.42×10^{-2}	4.22×10^{-2}
Whole dataset				
$\mathcal{L}_b$	1.52×10^{-2}	5.26×10^{-3}	3.27×10^{-2}	3.84×10^{-2}

the validation dataset provide a quantitative evaluation of generalization performance of tested models across unseen memristor dynamics. The best result of $5.77\cdot10^{-3}$ is obtained for the DualNN-MemODE model. The significant reduction of the average values of $\mathcal{L}_b$ for both proposed method when compared to existing methods demonstrates the enhanced modeling capability of the neural ODE framework.

The superior performance of the neural network model can be attributed to its capacity for learning complex nonlinear mappings directly from experimental data, whereas the MMS model relies on predetermined phenomenological relationships that may not fully capture the device-specific dynamics and material-dependent switching characteristics inherent in real memristor devices.

It should be noted that the proposed Neural ODE models, as well as the MMS and GMMS baselines, are deterministic in nature and therefore cannot capture the inherently stochastic aspects of memristor switching. As visible in Fig. 7(c), localised deviations between the predicted and measured hysteresis loops are most pronounced in the vicinity of the switching transitions, where cycle-to-cycle variability in the physical device is largest. This is an intrinsic limitation of any deterministic modelling framework applied to a stochastic device: the model converges to the mean trajectory, and residual errors in the switching region reflect the irreducible variance of the physical process rather than a systematic modelling deficiency. Moreover, the switching transitions occupy only a small fraction of the total measurement duration—the device spends the majority of each cycle in the quasi-static HRS or LRS regions, where the voltage changes slowly and the resistance remains approximately constant. Consequently, the number of samples falling within the narrow switching intervals is substantially smaller than the number of samples in the non-switching portions of the trajectory. The trajectory-averaged loss function $\mathcal{L}_b$ is therefore naturally dominated by the fitting quality in the more densely sampled regions, which may attenuate the contribution of the sparsely sampled switching transients. Despite this, the Neural ODE models achieve lower $\mathcal{L}_b$ values than MMS and GMMS indicating that the global fit is consistently superior even in the presence of these localised switching-region deviations.

While the neural ODE models employ a larger number of trainable parameters compared to the MMS and GMMS formulations, the improved accuracy cannot be attributed solely to increased model capacity. The key methodological distinction lies in the training paradigm: the proposed models are optimized via adjoint sensitivity methods, which enables simultaneous optimization of all parameters within a single, unified procedure. In contrast, the MMS and GMMS models require a two-stage optimization strategy—an initial computationally expensive global search over the parameter space followed by local gradient-based refinement—which is prone to sensitivity with respect to the initial parameter estimates and may converge to suboptimal solutions. Furthermore, the neural ODE framework eliminates the need for manually specified window functions, voltage threshold values, and switching rules.

The state variable dynamics are learned directly from experimental data without imposing assumptions about the functional form of the drift velocity or the boundary enforcement mechanism. This flexibility comes at the cost of reduced physical interpretability: while the MMS model parameters (e.g., R_{ON}, R_{OFF}, V_{ON}, V_{OFF}) carry direct physical meaning, the weights of a neural network do not admit such interpretation. However, the SingleNN-MemODE architecture partially preserves interpretability by retaining the physically motivated memductance formulation of Eq. (9), with R_{ON} and R_{OFF} optimized jointly with the network parameters.

Let us now compare computation times needed to find different models and evaluation times of models. Computations are carried out using a 3.5 GHz Intel i9-9900X processor. The median training duration for the SingleNN-MemODE and DualNN-MemODE architectures is approximately 11.3 minutes and 13.8 minutes, respectively. The median computational time necessary for parameter optimization of the MMS and GMMS models is approximately 20.4 minutes and 8.4 minutes, respectively. Add a comment that computation times for all methods are of the same order of magnitude and explain (if possible) why the more complex GMMS model has the shortest optimization time. The optimization times for all considered methods are of the same order of magnitude. The shorter optimization time of the GMMS model, despite its larger parameter space, can be attributed to better numerical conditioning of the forward problem — the MMS model more frequently encounters stiff regimes during integration, increasing the computational cost per function evaluation.

The computation time needed for the generation of 18 trajectories using the MMS and GMMS models is approximately 0.047 seconds and 0.051 seconds, respectively. In contrast, the DualNN-MemODE architecture needs approximately 3.97 seconds, while the SingleNN-MemODE architecture requires approximately 5.56 seconds to generate the same set of trajectories. The neural network models are integrated numerically using a relative error tolerance of 10^{-3} and an absolute error tolerance of 10^{-6}. The significantly longer evaluation time of the proposed architectures is a consequence of the computational cost of neural network

inference at each integration step of the ODE solver, whereas the analytical models evaluate inexpensive closed-form expressions. Add a comment that the evaluation time for the proposed method is much slower that for the existing methods.

B. HYPERPARAMETER SENSITIVITY ANALYSIS

Hyperparameter optimization is conducted to systematically explore the hyperparameter space and identify configurations that maximize the predictive performance of neural network architectures. The optimization employs the Tree-structured Parzen Estimator (TPE) algorithm via the `Optuna` framework [44]. TPE constructs probabilistic models of high- and low-performing configurations, sampling preferentially from promising regions while maintaining exploration diversity.

The search space encompasses both architectural and training-related hyperparameters. On the architectural side, the number of hidden layers is varied between one and five, with layer widths ranging from 16 to 512 neurons in increments of 16. Candidate activation functions includes widely used nonlinearities such as ReLU, GELU, SiLU, $\tanh$, ELU, leaky ReLU, sigmoid, identity and sine functions. For the output layer, the choice is restricted to linear, $\tanh$, sigmoid, and ReLU. The training-related search space included the learning rate, sampled on a logarithmic scale between 10^{-4} and 10^{-1}, batch size ranging from 1 to 18, optimizer type (Adam, SGD, AdamW, Nadam, AdaBelief), and parameters related to regularization and scheduling, including patience, cooldown, reduction factor, tolerance, and weight decay.

To quantify the relative influence of individual hyperparameters on the model performance, a surrogate-based importance analysis is performed. An XGBoost gradient boosting model [45] is trained on the explored configurations to predict validation loss. The surrogate model achieved an R^2 value of 0.99, indicating high fidelity in capturing the dependence of model performance on hyperparameter selection, where $\mathrm{R}^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$ represents the coefficient of determination measuring the proportion of explained variance, y_i are the observed values, $\hat{y}_i$ are the model predictions, and $\bar{y}$ is the mean of observed values. Importance is first estimated using the gain-based metric, which reflects the cumulative improvement in predictive accuracy attributed to each hyperparameter across the ensemble. Normalization ensures that scores sum up to unity, allowing for direct interpretation as relative contributions to performance variability.

As a complementary approach, permutation importance [46] is applied to the surrogate model. In this procedure, the values of each hyperparameter are permuted at random, and the resulting degradation in predictive accuracy is recorded. Repeated ten times for each feature, the mean error increase provides an alternative measure of importance, which, unlike gain-based metrics, also accounts for nonlinear dependencies and interactions among hyperparameters.

Results of the sensitivity analysis are presented in Fig. 8. According to the XGBoost Gain approach the dominant hyperparameters are the number of epochs, the gradient clipping value (the maximum allowable gradient norm), the ANN-f depth (the depth of the neural network used to approximate $f(\mathbf{x}, v)$), and the number of latent states, with the relative importance scores of 0.47, 0.27, 0.12, and 0.06, respectively. Collectively, the four most significant parameters account for more than 90% of the total observed performance variance.

The permutation importance approach identifies the following four most dominant hyperparameters: [approved]influencing model performance: the learning rate, the gradient clipping value, the number of epochs, and the ANN-$\mathcal{G}$ depth (the depth of the neural network used to approximate $\mathcal{G}(\mathbf{x})$). Their relative importance scores are 0.27, 0.24, 0.19, and 0.19, respectively.

Summarizing, the most important hyperparameters are identified as the number of epochs, the gradient clipping value, the learning rate, depths of neural networks and the number of latent states. Widths of neural networks, the batch size and the step length have moderate influence. Other hyperparameters have negligible effects of the model performance. For example, the choice of activation function exhibits a negligible effect (importance score below 0.01).

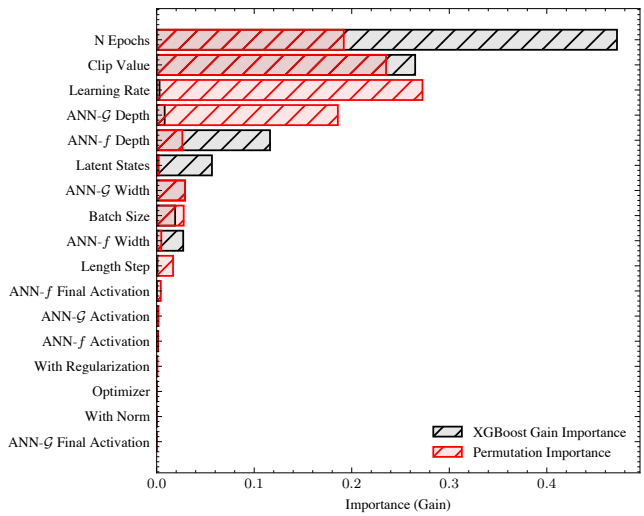

FIGURE 8. Hyperparameter importance for the DualNN-MemODE optimization task. Scores are computed using the XGBoost surrogate model and the permutation importance approach. Larger values indicate greater relative influence of the corresponding hyperparameter on the model performance. ANN-$\mathcal{G}$ and ANN-f refer to neural networks used to approximate $\mathcal{G}(\mathbf{x})$ and $f(\mathbf{x}, v)$, respectively.

The mean values of the validation loss obtained during the hyperparameter optimization procedure for selected hyperparameters are presented in Fig. 9. The figure summarizes the influence of the number of latent states, the optimizer type, and the activation functions employed in the two neural networks constituting the DualNN-MemODE architecture. For each case, the reported mean is computed by marginalizing over all remaining hyperparameters, while the corresponding 95% bootstrap confidence intervals are indicated by error bars. This analysis highlights the relative sensitivity of the model performance to discrete architectural and training-strategy choices. The results indicate that the number of

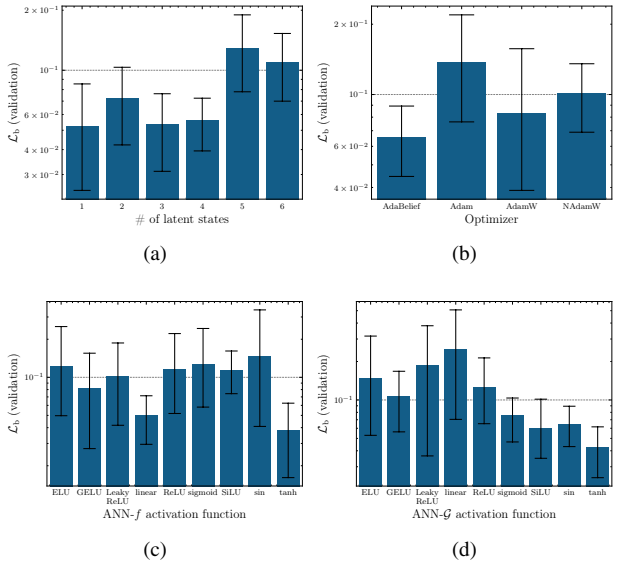

FIGURE 9. Validation loss obtained during hyperparameter optimization for selected hyperparameters: (a) number of latent states, (b) optimizer type for the first neural network, (c) activation function for the neural network used to approximate $\boldsymbol{f}(\mathbf{x}, \boldsymbol{v})$, and (d) activation function for the neural network used to approximate $\mathcal{G}(\mathbf{x})$. Mean values are computed by averaging over all other hyperparameters, with error bars indicating the 95% bootstrap confidence interval.

latent states has a moderate effect on model performance. Configurations with 1 to 4 latent states yield comparable validation loss, while increasing the number beyond 4 leads to a noticeable degradation, likely due to overfitting or increased optimization difficulty (Fig. 9a). Among the considered optimizers, AdaBelief achieves the lowest mean validation loss, while Adam performs notably worse (Fig. 9b). For the activation functions, $\tanh$ consistently yields the best results in both ANN-f and ANN-$\mathcal{G}$ networks (Fig. 9c,d). However, while these discrete choices produce visible differences in mean performance, their relative importance remains low compared to the dominant hyperparameters identified in Fig. 8 (number of epochs, gradient clipping, and learning rate), which together account for the majority of performance variance.

What are the conlusions from this analysis and results shown in Fig. 9. Either add some explanations or remove. And first of all explain it to me :-).

IV. CONCLUSIONS

This work presented two novel neural ODE-based architectures for modeling the dynamics of memristor devices. The proposed models utilize the power of neural networks to capture complex, nonlinear behaviors inherent in memristive systems.

The proposed models were applied for modeling of real Self-Directed Channel (SDC) memristors. Extensive evaluation against experimental data demonstrates that the neural ODE approach significantly outperforms traditional phenomenological models, such as the MMS and GMMS models, in terms of accuracy and fidelity. The results highlight the capability of the neural ODE approach to learn intricate device dynamics directly from data, thereby providing a powerful tool for simulating memristor behavior. Hyperparameter sensitivity analysis further elucidates the critical factors influencing model performance, guiding future optimization efforts. Obtained results show that the computation time needed to optimize the proposed models is comparable to that of the existing models, with all methods requiring optimization times of the same order of magnitude. On the other hand, the evaluation time needed to compute trajectories using the proposed models is significantly larger than for the MMS and GMMS models, due to the computational cost of neural network inference at each integration step of the ODE solver.

We stress that all experimental data used in this study were acquired from a single physical SDC device. The proposed Neural ODE training pipeline—including adjoint sensitivity-based optimization and the multi-component loss function—may be applied to other memristor technologies (e.g., HfO_2-based RRAM, phase-change memory). Verification whether this approach would be successfull in modeling of other types of memristive devices remains to be validated and constitutes an important direction for future work.

The findings of this study underscore the potential of neural ODEs as a versatile and effective framework for modeling complex physical systems, with applications in the design and analysis of memristor-based circuits. Developed methods pave the way to more accurate and efficient simulation tools, facilitating advancements in memristor technology and data-driven modeling of nonlinear dynamical systems.

REFERENCES

[1] D. B. Strukov, G. S. Snider, D. R. Stewart, and R. S. Williams, "The missing memristor found," *Nature*, vol. 453, no. 7191, pp. 80–83, May 2008.

[2] C. Yakopcic, T. M. Taha, G. Subramanyam, R. E. Pino, and S. Rogers, "A memristor device model," *IEEE Electron Device Lett.*, vol. 32, no. 10, pp. 1436–1438, Oct. 2011.

[3] S. Kvatinsky, E. G. Friedman, A. Kolodny, and U. C. Weiser, "TEAM: ThrEshold Adaptive Memristor Model," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 60, no. 1, pp. 211–221, Jan. 2013.

[4] S. Kvatinsky, M. Ramadan, E. G. Friedman, and A. Kolodny, "VTEAM: A general model for voltage-controlled memristors," *IEEE Trans. Circuits Syst. II, Express Briefs*, vol. 62, no. 8, pp. 786–790, 2015.

[5] P. Sheridan, K.-H. Kim, S. Gaba, T. Chang, L. Chen, and W. Lu, "Device and SPICE modeling of RRAM devices," *Nanoscale*, vol. 3, no. 9, p. 3833, 2011.

[6] X. Guan, S. Yu, and H.-S. P. Wong, "A SPICE compact model of metal oxide resistive switching memory with variations," *IEEE Electron Device Lett.*, vol. 33, no. 10, pp. 1405–1407, Oct. 2012.

[7] P.-Y. Chen and S. Yu, "Compact modeling of RRAM devices and its applications in 1T1R and 1S1R array design," *IEEE Trans. Electron Devices*, vol. 62, no. 12, pp. 4022–4028, Dec. 2015.

[8] M. D. Marco, M. Forti, L. Pancioni, G. Innocenti, A. Tesi, and F. Corinto, "Oscillatory circuits with a real non-volatile Stanford memristor model," *IEEE Access*, vol. 10, pp. 13 650–13 662, 2022.

[9] J. Secco, F. Corinto, and A. Sebastian, "Flux–charge memristor model for phase change memory," *IEEE Trans. Circuits Syst. II, Express Briefs*, vol. 65, no. 1, pp. 111–114, 2018.

[10] Y. Zhang, X. Wang, Y. Li, and E. G. Friedman, "Memristive model for synaptic circuits," *IEEE Trans. Circuits Syst. II, Express Briefs*, vol. 64, no. 7, pp. 767–771, 2017.

[11] L. Xie, J. Shen, P. Feng, A. Mifsud, A. Malik, A. Nassibi, V. Manatchinapisit, and C. Papavassiliou, "A data-driven stochastic memristor model

for integrated circuit simulation," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 6, pp. 2300–2313, Jun. 2025.

[12] X. Wang, B. Xu, and L. Chen, "Efficient memristor model implementation for simulation and application," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 36, no. 7, pp. 1226–1230, Jul. 2017.

[13] T. Kahale and D. Tannir, "Memristor modeling using the modified nodal analysis approach," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 4, pp. 1191–1195, Apr. 2022.

[14] N. Lupo, E. Pérez, C. Wenger, F. Maloberti, and E. Bonizzoni, "Analysis of parasitic effects in filamentary-switching memristive memories using an approximated verilog-a memristor model," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 5, pp. 1935–1947, 2019.

[15] C. Yakopcic, T. M. Taha, D. J. Mountain, T. Salter, M. J. Marinella, and M. McLean, "Memristor model optimization based on parameter extraction from device characterization data," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 5, pp. 1084–1095, May 2020.

[16] T. W. Molter and M. A. Nugent, "The generalized metastable switch memristor model," in *CNNA 2016; 15th Int. Workshop on Cellular Nanoscale Networks and their Applications*, 2016, pp. 1–2.

[17] E. Linn, A. Siemon, R. Waser, and S. Menzel, "Applicability of well-established memristive models for simulations of resistive switching devices," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 61, no. 8, pp. 2402–2410, 2014.

[18] Z. Galias, "Study of amplitude control and dynamical behaviors of a memristive band pass filter circuit," *IEEE Trans. Circuits Syst. II, Express Briefs*, vol. 65, no. 5, pp. 637–641, May 2018.

[19] Y. Li, C. Li, S. Zhang, Y. Zheng, and G. Chen, "Offset boosting-oriented construction of multi-scroll attractor via a memristor model," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 2, pp. 918–931, 2025.

[20] L. Minati, L. Gambuzza, W. Thio, J. Sprott, and M. Frasca, "A chaotic circuit based on a physical memristor," *Chaos, Solitons Fractals*, vol. 138, p. 109990, Sep. 2020.

[21] V. Ostrovskii, P. Fedoseev, Y. Bobrova, and D. Butusov, "Structural and parametric identification of Knowm memristors," *Nanomaterials*, vol. 12, no. 1, p. 63, Dec. 2021.

[22] G. A. Laguna-Sanchez, M. Lopez-Guerrero, and R. Barron-Fernandez, "The probabilistic behavior of the set and reset thresholds in Knowm's SDC memristors: Characterization and simulation," *IEEE Latin America Transactions*, vol. 21, no. 12, pp. 1266–1274, Dec. 2023.

[23] V. A. Slipko and Y. V. Pershin, "Importance of the window function choice for the predictive modelling of memristors," *IEEE Trans. Circuits Syst. II, Express Briefs*, vol. 68, no. 6, pp. 2167–2171, 2021.

[24] F. Corinto and A. Ascoli, "A boundary condition-based approach to the modeling of memristor nanostructures," *IEEE Trans. Circuits Syst. I*, vol. 59, no. 11, pp. 2713–2726, 2012.

[25] Z. Galias, "Simulations of memristors using the Poincaré map approach," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 3, pp. 963–971, Mar. 2020.

[26] V. Ntinas, A. Ascoli, I. Messaris, Y. Wang, V. Rana, S. Menzel, and R. Tetzlaff, "Toward simplified physics-based memristor modeling of valence change mechanism devices," *IEEE Trans. Circuits Syst. II, Express Briefs*, vol. 69, no. 5, pp. 2473–2477, 2022.

[27] D. Hajtó, W. El-Geresy, D. Gündüz, and G. Cserey, "State characterisation of self-directed channel memristive devices," *Eng. Res. Express*, vol. 7, no. 4, p. 045391, dec 2025.

[28] Y. Lee, K. Kim, and J. Lee, "A compact memristor model based on physics-informed neural networks," *Micromachines*, vol. 15, no. 2, 2024.

[29] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," in *32nd Conf. Neural Information Processing Systems (NeurIPS 2018)*, Montréal, Canada, 2018.

[30] K. A. Campbell, "Self-directed channel memristor for high temperature operation," *Microelectron. J.*, vol. 59, pp. 10–14, Jan. 2017.

[31] B. Garda and K. Bednarz, "Comprehensive study of SDC memristors for resistive RAMapplications," *Energies*, vol. 17, no. 2, p. 467, Jan. 2024.

[32] K. Bednarz and B. Garda, "Measurement and modeling of self-directed channel (SDC) memristors: An extensive study," *Energies*, vol. 17, no. 21, p. 5400, Oct. 2024.

[33] L. Chua and S. M. Kang, "Memristive devices and systems," *Proceedings of the IEEE*, vol. 64, no. 2, pp. 209–223, 1976.

[34] R. Cipolla, Y. Gal, and A. Kendall, "Multi-task learning using uncertainty to weigh losses for scene geometry and semantics," in *2018 IEEE/CVF Conf. Computer Vision and Pattern Recognition*. IEEE, Jun. 2018, pp. 7482–7491.

[35] Z. Chen, V. Badrinarayanan, C.-Y. Lee, and A. Rabinovich, "GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks," in *Proceedings of the 35th Int. Conf. Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, Jul. 2018, pp. 794–803.

[36] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[37] P. Kidger and C. Garcia, "Equinox: neural networks in JAX via callable PyTrees and filtered transformations," *Differentiable Programming workshop at Neural Information Processing Systems 2021*, 2021.

[38] P. Stumm and A. Walther, "New algorithms for optimal online checkpointing," *SIAM J. Sci. Comput.*, vol. 32, no. 2, pp. 836–854, Jan. 2010.

[39] C. Tsitouras, "Runge–Kutta pairs of order 5(4) satisfying only the first column simplifying assumption," *Comput. Math. Appl.*, vol. 62, no. 2, pp. 770–775, Jul. 2011.

[40] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD Int. Conf. Knowledge Discovery Data Mining*, ser. KDD'19. ACM, Jul. 2019, pp. 2623–2631.

[41] R. Feldt, "Blackboxoptim.jl: Black-box optimization for Julia," 2013–2025.

[42] C. Zhu, R. H. Byrd, P. Lu, and J. Nocedal, "Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization," *ACM Trans. Math. Software*, vol. 23, no. 4, pp. 550–560, Dec. 1997.

[43] S. Arlot and A. Celisse, "A survey of cross-validation procedures for model selection," *Statistics Surveys*, vol. 4, no. none, Jan. 2010.

[44] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *The 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.

[45] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD Int. Conf. Knowledge Discovery Data Mining*, ser. KDD '16. ACM, Aug. 2016, pp. 785–794.

[46] A. Altmann, L. Toloşi, O. Sander, and T. Lengauer, "Permutation importance: a corrected feature importance measure," *Bioinformatics*, vol. 26, no. 10, pp. 1340–1347, Apr. 2010.